Project Report: Design and Simulation of a VHDL-Based Digital Frequency & Duty Cycle Meter

Sai Vishal Reddy Malireddy
Department of Electrical Engineering
Indian Institute of Technology Bombay

July 2026

1 Project Overview

This report details the design, implementation, and simulation of a fully digital Frequency and Duty Cycle Meter using VHDL. The system is designed to accurately measure an unknown asynchronous periodic signal by leveraging a 50 MHz system clock.

Unlike microcontroller-based software timers, this system is designed purely in digital logic (Register Transfer Level), allowing for parallel execution and exact clock-cycle precision. The design emphasizes robust digital principles, including input synchronization to mitigate metastability and a dedicated Finite State Machine (FSM) for precise timing control.

2 Objectives

The primary objective of this project is to build a robust, synthesizable hardware meter capable of analyzing a raw input signal and outputting two key parameters:

1. **Frequency (Hz):** The number of cycles per second.
2. **Duty Cycle (%):** The ratio of the signal's high-time to its total period.

3 Theoretical Framework

3.1 Frequency Measurement (Fixed Gate Method)

The system calculates frequency by opening a precise 1-second measurement window (the "gate") and counting the number of rising edges of the unknown signal that occur within that window. Because the gate is exactly 1 second, the accumulated count directly equals the frequency in Hertz.

3.2 Duty Cycle Measurement

Duty cycle is calculated using the fast 50 MHz reference clock rather than the fixed gate. During a single period of the input signal, two parallel counters accumulate data:

- **Period Count:** Total system clock ticks between two consecutive input rising edges.
- **High-Time Count:** System clock ticks strictly while the input signal is logic HIGH.

The final percentage is calculated as:

$$\text{Duty Cycle} = \left(\frac{\text{High\_Time}}{\text{Period}} \right) \times 100 \tag{1}$$

3.3 Metastability and Synchronization

Because the input signal originates from an external source, it is asynchronous to the internal 50 MHz clock domain. Feeding this directly into synchronous counters risks metastability—a state where flip-flops hover between logic levels, causing unpredictable behavior or system crashes. To prevent this, the raw input is passed through a 2-stage D-flip-flop synchronizer before any edge detection or counting occurs.

4 System Architecture

The top-level hardware is strictly modular and integrates four primary subsystems.

1. **Input Conditioning Block:** Consists of the 2-stage synchronizer and an edge detector. The edge detector compares the current synchronized state with the previous clock cycle's state to generate a 1-clock-cycle wide pulse upon detecting a rising edge.
2. **Timebase (Clock Divider):** A sequential counter that scales the 50 MHz system clock down to a 1 Hz square wave, providing the exact 1-second gate required for frequency counting.
3. **Data Acquisition (Counters):** Three instances of a generic configurable 32-bit counter module. One tracks the input edges (Frequency), one tracks the total system ticks (Period), and one tracks the active-high ticks (High-Time).
4. **Output Latch & Calculator:** A synchronous register block that captures the final counter values when triggered by the FSM, performs the duty cycle percentage division, and holds the data stable on the output ports.

5 Finite State Machine (FSM) Design

The control logic is governed by a Moore-style FSM that dictates the timing of resets, counter enables, and data latching.

- **IDLE:** The system holds all counter resets HIGH. Upon detecting the first valid input edge, it transitions to the `WAIT_EDGE` state.
- **WAIT_EDGE:** The system waits for the clock divider to drive the 1-second gate tick HIGH, ensuring the measurement starts precisely at the beginning of a time window.
- **MEASURE:** The counter enable lines are driven HIGH. The frequency, period, and high-time counters actively accumulate data.

- **CAPTURE:** Triggered when the 1-second gate closes. The FSM drops the enable lines and asserts the latch enable signal for exactly one clock cycle to capture the final counts.
- **HOLD:** The FSM waits for the clock divider to finish its cycle before returning to `IDLE` to begin a new measurement, keeping the output display stable.

6 Simulation and Verification

The design was verified using a self-checking VHDL testbench, analyzed and elaborated in Intel Quartus Prime for RTL simulation.

6.1 Test Methodology

A testbench was developed to instantiate the system (Device Under Test) and drive it with a simulated 50 MHz clock. A synthetic asynchronous test signal was injected into the input port with a 1 kHz frequency (1.0 ms period) and a 25% duty cycle (0.25 ms HIGH, 0.75 ms LOW).

6.2 Simulation Results

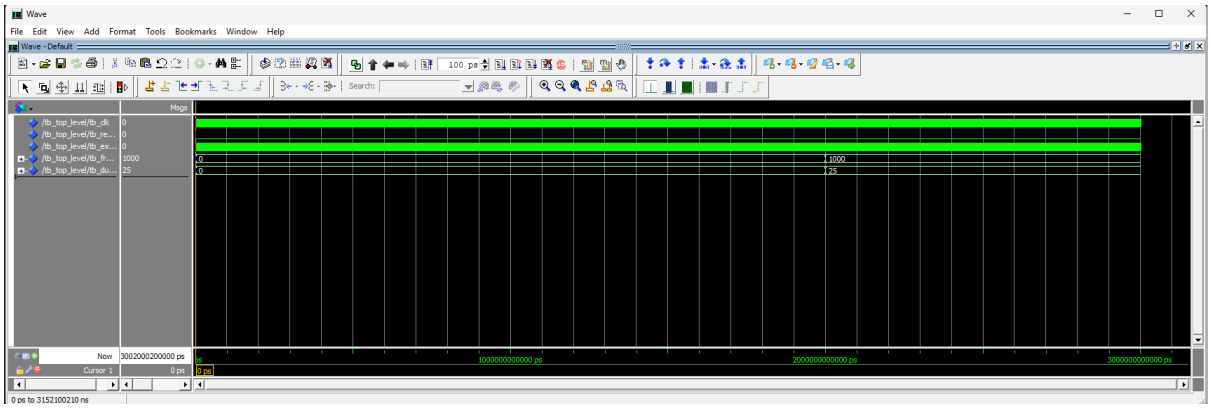

Figure 1: ModelSim RTL Simulation Waveform

As shown in the waveform capture, the FSM successfully transitions from the `MEASURE` state to the `CAPTURE` state. At exactly the boundary of the measurement gate, the internal latch signal pulses HIGH. On the subsequent clock edge, the output registers successfully update:

- Frequency latches to 1000 (representing 1 kHz).
- Duty Cycle latches to 25 (representing 25%).

The 2-stage synchronizer successfully debounced the input, and no metastable states propagated through the FSM.

7 Conclusion & Future Scope

The digital frequency and duty cycle meter was successfully designed and verified at the RTL level. The system proves highly accurate for standard signals due to the 50 MHz reference resolution and robust synchronization protocols.

Future Improvements for Hardware Deployment:

1. **Auto-Ranging:** The current fixed-gate method is excellent for high frequencies but loses precision for ultra-low frequencies (e.g., < 10 Hz). Implementing an auto-ranging FSM that dynamically switches to a reciprocal period measurement would vastly improve low-frequency accuracy.

2. **Hardware Division:** The integer division used in the duty cycle math block is ideal for simulation but highly resource-intensive for physical synthesis. Before flashing this to an FPGA development board, a dedicated Division IP Core or a shift-and-subtract state machine should be integrated to optimize silicon gate usage.